

Assignment No.

07:

Simulation using

Synopsys VCS

1. Design and simulate a 4-bit Universal Shift Register using Verilog HDL and Synopsys VCS.

a) Implement Hold, Shift Left, Shift Right, and Parallel Load operations.

b) Develop a suitable testbench to verify all operating modes.

c) Generate a VCD waveform file during simulation.

d) Verify the generated waveform using GTKWave and capture the simulation results.

DESIGN CODE:

```
module usr_4bit (  
    input wire clk,  
    input wire rst_n,  
    input wire [1:0] select,  
    input wire shift_left_in,  
    input wire shift_right_in,  
    input wire [3:0] d,  
    output reg [3:0] q    );  
  
    always @(posedge clk or negedge rst_n) begin  
        if (!rst_n) begin  
            q <= 4'b0000;  
        end else begin  
            case (select)  
                2'b00: q <= q;  
                2'b01: q <= {shift_right_in, q[3:1]};  
                2'b10: q <= {q[2:0], shift_left_in};  
                2'b11: q <= d;  
                default: q <= q;  
            endcase  
        end  
    end  
  
endmodule
```

TESHBENCH CODE:

```
`timescale 1ns/1ps  
  
module tb_usr_4bit;  
  
    reg clk;  
    reg rst_n;  
    reg [1:0] select;  
    reg shift_left_in;
```

```

reg shift_right_in;
reg [3:0] d;
wire [3:0] q;

usr_4bit uut (
    .clk(clk),
    .rst_n(rst_n),
    .select(select),
    .shift_left_in(shift_left_in),
    .shift_right_in(shift_right_in),
    .d(d),
    .q(q));

always #10 clk = ~clk;

initial begin

    $dumpfile("usr_simulation.vcd");
    $dumpvars(0, tb_usr_4bit);

    clk = 0;
    rst_n = 0;
    select = 2'b00;
    shift_left_in = 0;
    shift_right_in = 0;
    d = 4'b0000;

    #15 rst_n = 1;

    #10;
    select = 2'b11;
    d = 4'b1010;

    #20;
    select = 2'b00;
    d = 4'b1111;

    #20;
    select = 2'b01;
    shift_right_in = 1;
    #20;
    shift_right_in = 0;
    #40;

    #20;
    select = 2'b10;
    shift_left_in = 1;

```

```

#20;
shift_left_in = 0;
#40;

$display("Simulation complete. Waveform file generated.");
$finish;

end
endmodule

```

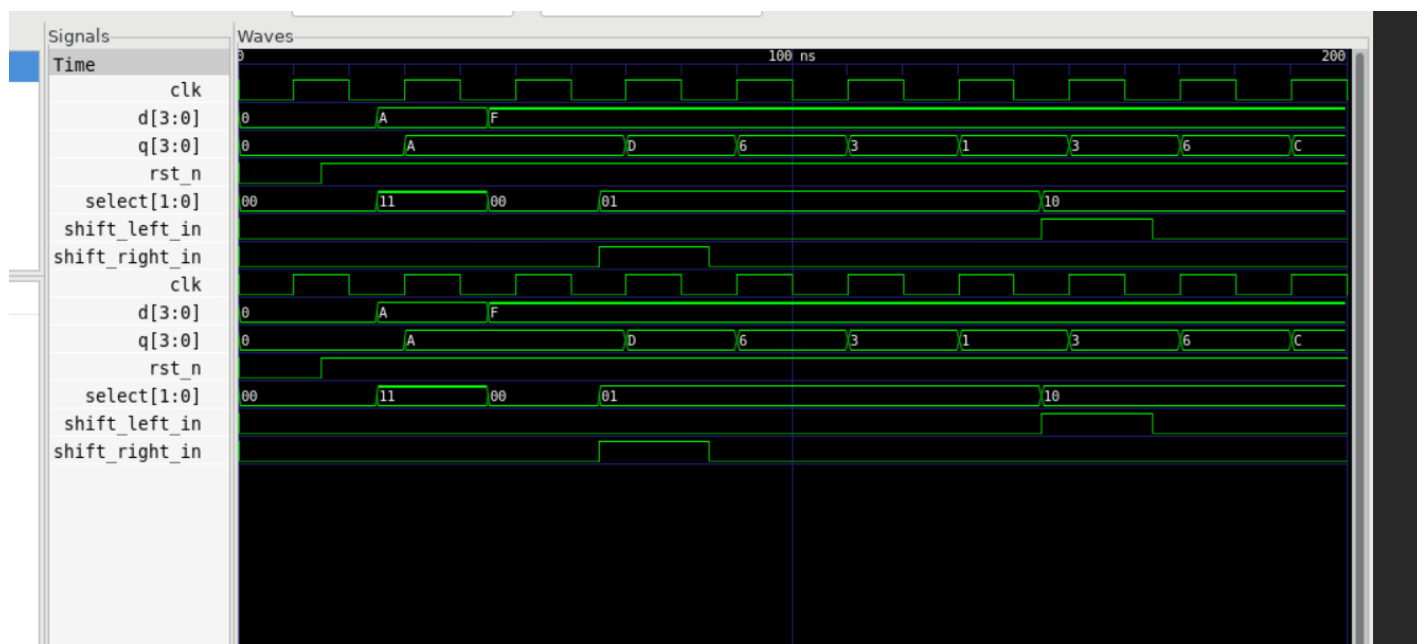
VCD WAVE FORM:

```

Compiler version W-2024.09; Runtime version W-2024.09; Jul 17 20:44 2026
[SCL] SCL trace is enabled.
Checkout succeeded: VCS-BASE-RUNTIME/AF3AD4D7787CF4828637
License file: 27020@14.139.1.126
License Server: 27020@14.139.1.126
[SCL] 07/17/2026 20:44:34 Checking status for feature VCS-BASE-RUNTIME
[SCL] 07/17/2026 20:44:34 PID:210911 Client:smartlab6.calicut.nielit.in Server:27020@14.139.1.126 Checkout succ
RUNTIME 2024.09 from VCS-Base-Runtime-Pkg
 0 | 0 | 00 | 0 | 0 | 0000 | 0000
15 | 1 | 00 | 0 | 0 | 0000 | 0000
25 | 1 | 11 | 0 | 0 | 1010 | 0000
30 | 1 | 11 | 0 | 0 | 1010 | 1010
45 | 1 | 00 | 0 | 0 | 1111 | 1010
65 | 1 | 01 | 0 | 1 | 1111 | 1010
70 | 1 | 01 | 0 | 1 | 1111 | 1101
85 | 1 | 01 | 0 | 0 | 1111 | 1101
90 | 1 | 01 | 0 | 0 | 1111 | 0110
110 | 1 | 01 | 0 | 0 | 1111 | 0011
130 | 1 | 01 | 0 | 0 | 1111 | 0001
145 | 1 | 10 | 1 | 0 | 1111 | 0001
150 | 1 | 10 | 1 | 0 | 1111 | 0011
165 | 1 | 10 | 0 | 0 | 1111 | 0011
170 | 1 | 10 | 0 | 0 | 1111 | 0110
190 | 1 | 10 | 0 | 0 | 1111 | 1100
Simulation complete.Waveform file generated
$finish called from file "../RTL/TB/tb_usr_4bit.v", line 66.
$finish at simulation time 205000
VCS Simulation Report
Time: 205000 ps
CPU Time: 0.250 seconds; Data structure size: 0.0Mb
Fri Jul 17 20:44:34 2026

```

GTKWave:



Q2. Design and simulate an 8-bit Arithmetic Logic Unit (ALU) using Verilog HDL and Synopsys VCS.

a) Implement the following operations:

Addition

Subtraction

Bitwise

AND Bitwise

OR Bitwise

XOR

Logical Left Shift

Logical Right Shift

b) Develop a comprehensive testbench to verify all ALU operations.

c) Generate the simulation waveform file (.vcd).

d) Analyze the generated waveform using GTKWave and verify the correctness of each operation.

DESIGN CODE:

```
`timescale 1ns/1ps
module alu_8bit (
    input wire [7:0] a,
    input wire [7:0] b,
    input wire [2:0] opcode,
    output reg [7:0] alu_out,
    output reg      carry_out );

    always @(*) begin

        alu_out    = 8'b0000_0000;
        carry_out  = 1'b0;

        case (opcode)
            3'b000: begin
                {carry_out, alu_out} = a + b;
            end
            3'b001: begin
                alu_out = a - b;
                carry_out = (a < b);
            end
            3'b010: begin
                alu_out = a & b;
            end
            3'b011: begin
                alu_out = a | b;
            end
        endcase
    end
endmodule
```

```

        end
        3'b100: begin
            alu_out = a ^ b;
        end
        3'b101: begin
            alu_out = a << 1;
        end
        3'b110: begin
            alu_out = a >> 1;
        end
        3'b111: begin
            alu_out = a;
        end
        default: begin
            alu_out = a;
        end
    endcase
end
endmodule

```

TESTBENCH CODE:

```

`timescale 1ns/1ps

module tb_alu_8bit;

    reg [7:0] a;
    reg [7:0] b;
    reg [2:0] opcode;
    wire [7:0] alu_out;
    wire carry_out;

    alu_8bit uut (
        .a(a),
        .b(b),
        .opcode(opcode),
        .alu_out(alu_out),
        .carry_out(carry_out) );

    initial begin
        $display("\n-----");
        $display("   Time(ns) | Opcode |   Input A   |   Input B   | Out");
Result | Carry");
        $display("-----");
        $monitor("%11d |   %b   |   %b   |   %b   |   %b   |   %b",
            $time, opcode, a, b, alu_out, carry_out);
    end
endmodule

```

```
end
```

```
initial begin
```

```
    $dumpfile("alu_simulation.vcd");
```

```
    $dumpvars(0, tb_alu_8bit);
```

```
    a = 8'h00; b = 8'h00; opcode = 3'b111;
```

```
    #10;
```

```
    opcode = 3'b000;
```

```
    a = 8'hF0; b = 8'h0F;
```

```
    #10;
```

```
    a = 8'hFF; b = 8'h01;
```

```
    #10;
```

```
    opcode = 3'b001;
```

```
    a = 8'h45; b = 8'h10;
```

```
    #10;
```

```
    a = 10;    b = 20;
```

```
    #10;
```

```
    opcode = 3'b010;
```

```
    a = 8'b1100_1010; b = 8'b1010_0101;
```

```
    #10;
```

```
    opcode = 3'b011;
```

```
    a = 8'b1100_1010; b = 8'b1010_0101;
```

```
    #10;
```

```
    opcode = 3'b100;
```

```
    a = 8'b1100_1010; b = 8'b1010_0101;
```

```
    #10;
```

```
    opcode = 3'b101;
```

```
    a = 8'b1000_0011;
```

```
    #10;
```

```
    opcode = 3'b110;
```

```
    a = 8'b1000_0011;
```

```
    #10;
```

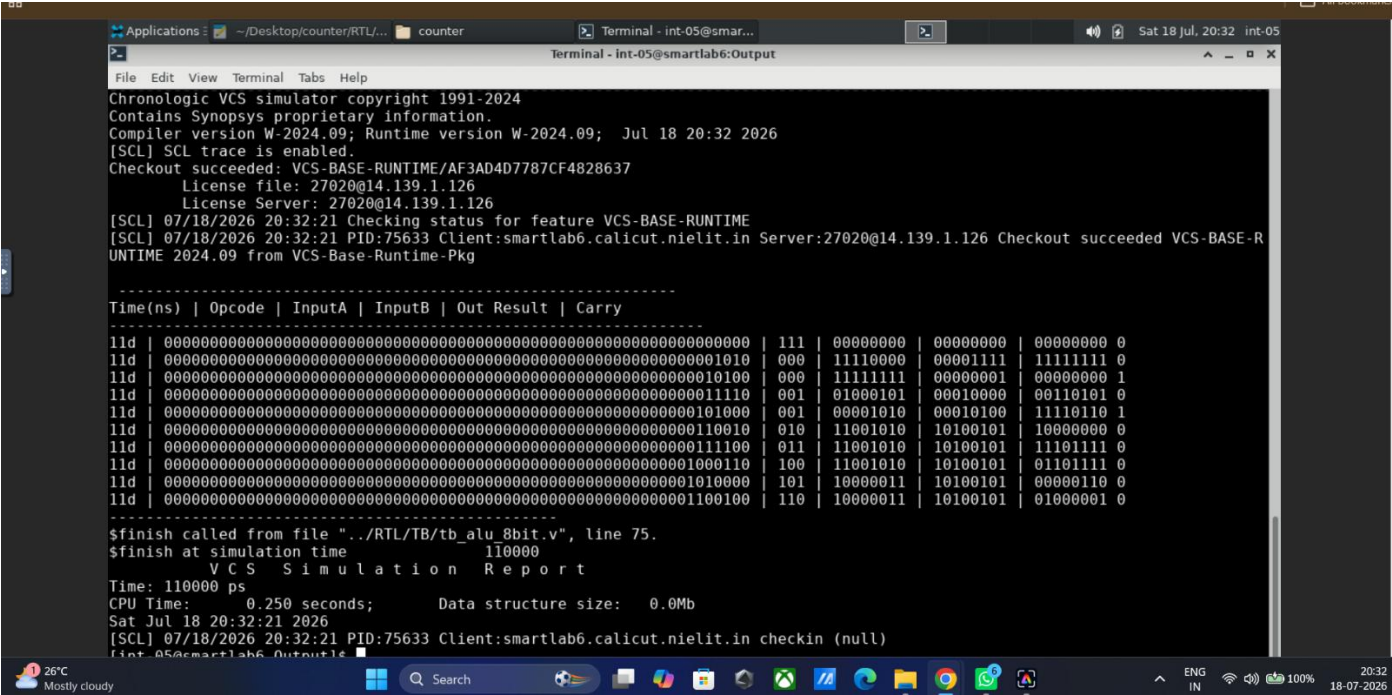
```
    $display("-----");
```

```
    $finish;
```

```
end
```

```
endmodule
```

VCD FILE :



GTKWave:

